

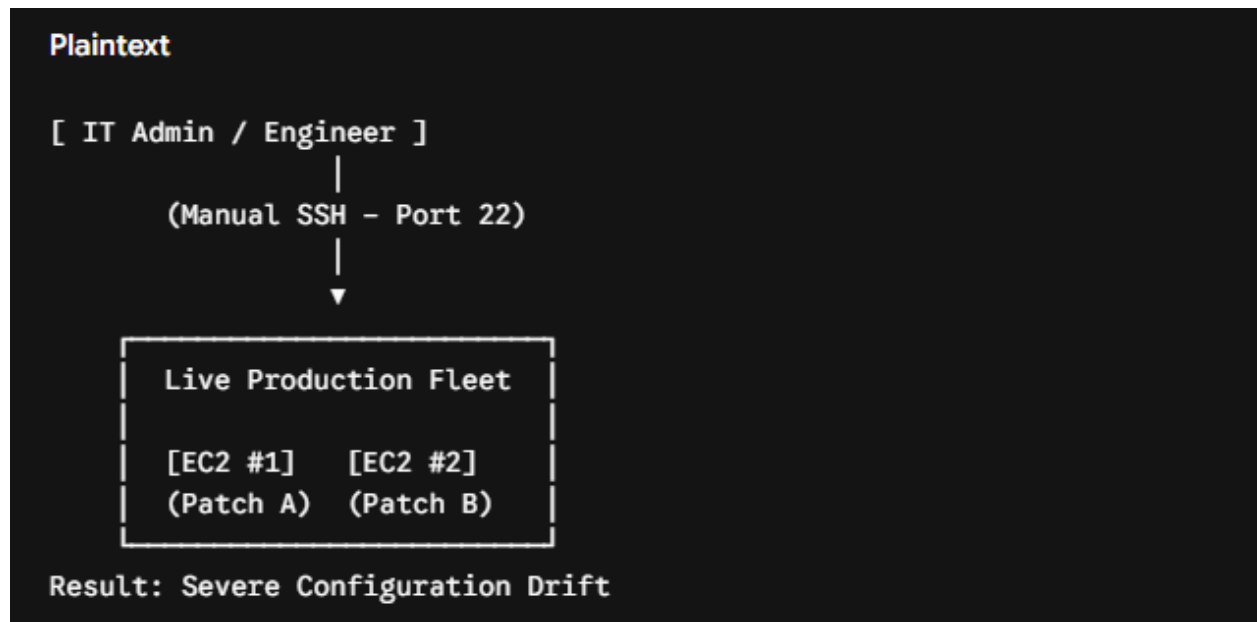
Case Study 3: Immutable Infrastructure & Disaster Recovery at Scale

1. Executive Summary & Business Context

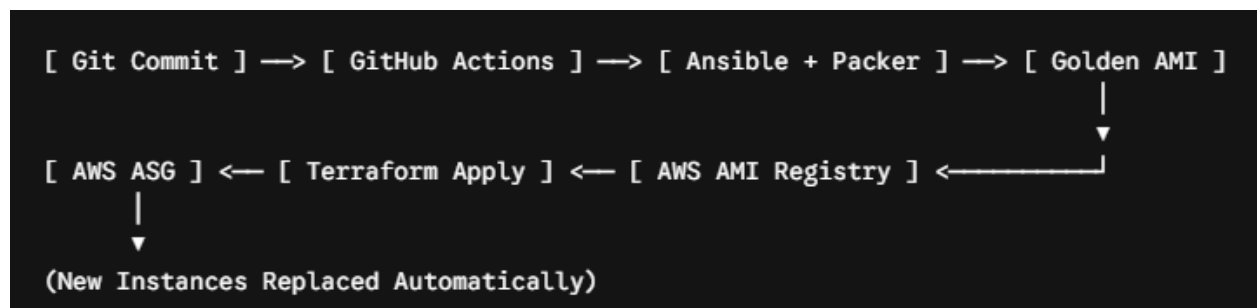
- **Company Profile:** A premier fintech organization processing stock trading, payment processing, and loan underwriting solutions.
- **The Trigger Event:** During a routine off-hours security patching exercise, system administrators manually executed apt-get upgrade and modified PAM authentication configurations directly on 120 production Linux servers via SSH. A typo in a configuration file (/etc/pam.d/common-auth) went unnoticed. The following morning, when a secondary node failed and the load balancer redirected primary traffic to the patched nodes, user authentication failed globally. The disaster recovery playbook required engineers to manually SSH into each server to debug differences, resulting in **14 hours of total system downtime**.
- **Business Impact:** 5.8 million in SLA non-compliance penalties, regulatory scrutiny from financial authorities, and severe reputational damage.
- **Strategic Mandate:** Mandate an **Immutable Infrastructure** model across all cloud resources. Manual SSH access (port 22) to production environments is strictly revoked. Infrastructure must be defined as code, built into immutable machine images, and fully deployable to an alternate AWS region in under 15 minutes.

2. Legacy Architecture vs. Target Architecture

Legacy Manual Infrastructure (Configuration Drift)



Target Immutable Infrastructure Pipeline



Architectural Analysis & Flaws

- **Legacy Manual Infrastructure (Configuration Drift)**
 - **The Problem:** In a traditional mutable setup, administrators apply patches live. Over time, manual edits cause severe configuration drift where no two servers in a fleet share identical software versions or Linux kernels.
 - **Technical Flaws:**
 1. **Configuration Drift:** Months of manual patches mean production servers have diverged completely.
 2. **Untracked Security Vulnerabilities:** Emergency fixes applied directly to live

servers are rarely committed back to source control, meaning rebuilding a node from scratch produces a server missing vital patches.

3. **High Recovery Time Objective (RTO):** Rebuilding infrastructure during a regional outage requires manual server provisioning and script execution, dragging recovery out for hours or days.

- **Target Immutable Infrastructure Pipeline**

- **The Modern Workflow:** Code changes are committed to Git → Triggering a GitHub Actions pipeline → Running Ansible playbooks via Packer to bake a Golden AMI → Pushing the image to the AWS AMI Registry → Applying Terraform updates to an AWS Auto Scaling Group (ASG) which performs a rolling instance refresh, terminating obsolete instances automatically.

3. Tool-by-Tool Implementation Breakdown

A. Linux (Operating System Layer)

- **Role:** Baseline OS Layer & System Hardening Target.
- **Implementation Details:**
 - Enterprise Linux (Ubuntu LTS / Rocky Linux) configured according to CIS (Center for Internet Security) Benchmarks.
 - Direct root logins are disabled (PermitRootLogin no in /etc/ssh/sshd_config).
 - SSH service (sshd) is entirely disabled or blocked by Security Group ingress rules in production environments.
 - Core OS audit logging (auditd) is configured to send kernel logs directly to AWS CloudWatch for immutable log auditing.

B. Git & GitHub (Configuration Audit Trail)

- **Role:** Single Source of Truth for System Configuration.
- **Implementation Details:**
 - Operating system parameters, package lists, firewall settings, and application dependencies are declared inside Ansible repositories.
 - Any infrastructure or OS modification requires a Pull Request (PR) with mandatory approval from Security and Infrastructure leads, ensuring a **100%** auditable history of system changes.

C. Ansible (System Configuration & Image Hardening)

- **Role:** Automated OS Provisioning & "Image Baking".
- **Implementation Details:**
 - Ansible playbooks execute during image construction (using HashiCorp Packer or AWS Image Builder) on temporary build instances.
 - Playbooks execute idempotently to update OS packages, install runtime

dependencies (e.g., Python, Docker, monitoring agents), configure system systemd services, and remove build-time temporary files/ssh keys.

YAML

```
# Example Ansible Playbook Excerpt for OS Hardening
- name: Hardening Production Linux Image
  hosts: all
  become: yes
  tasks:
    - name: Update all system packages
      apt:
        name: "*"
        state: latest
        update_cache: yes

    - name: Install mandatory security and monitoring tools
      apt:
        name:
          - fail2ban
          - ufw
          - amazon-cloudwatch-agent
        state: present

    - name: Disable SSH Password Authentication
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PasswordAuthentication'
        line: 'PasswordAuthentication no'
```

D. AWS (Infrastructure Hosting Layer)

- **Role:** Elastic Cloud Hosting & Image Storage.
- **Implementation Details:**
 - **AMI (Amazon Machine Image):** Serves as the immutable snapshot containing the pre-configured OS, dependencies, and application code.
 - **AWS Auto Scaling Group (ASG):** Manages instance lifecycles. Instance replacement uses Instance Refresh policies with configurable warm-up times, replacing old nodes with new ones seamlessly.
 - **AWS Systems Manager (SSM) Session Manager:** Completely replaces port 22 SSH. If emergency debugging is required, engineers authenticate via IAM to open interactive

sessions logged to CloudWatch.

E. Terraform (Infrastructure Provisioning)

- **Role:** Declarative Deployment & Environment Orchestration.
- **Implementation Details:**
 - Terraform references the newly baked AMI ID using dynamic data sources or input variables.
 - Terraform updates the Launch Template referenced by the Auto Scaling Group.
 - In the event of a total regional disaster (e.g., us-east-1 failure), changing a single region variable in Terraform and running terraform apply recreates the entire production VPC, subnet layout, load balancers, and EC2 fleets in us-west-2 within minutes.

4. Key Architectural Trade-offs & Decisions

Decision Area	Mutable Infrastructure (Traditional)	Immutable Infrastructure (Modern)	Trade-off / Architectural Rationale
Patching Speed	Fast for minor fixes (run SSH command in seconds).	Slower initial deployment (must bake AMI & run pipeline: 10-15 mins).	Trading instant inline patching speed for absolute operational predictability and security.
Testing Parity	Low. Production drift causes patches to behave differently across hosts.	100% Parity. The exact AMI deployed to Staging is deployed to Production.	Eliminates "worked in staging, failed in production" bugs.
Storage Cost	Low. Existing servers are continuously overwritten.	Low-to-Moderate. Storing versioned AMIs incurs minimal S3 backend costs.	Minimal storage cost is negligible compared to the cost of system downtime.